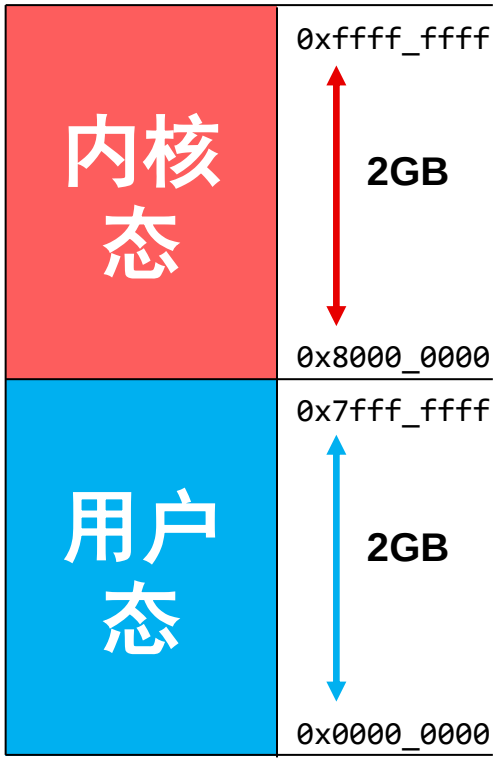
32位MIPS的内存分配

- 内存空间分配
  - 32位的地址决定了能管理的内存最大为2^{32}=4GB的大小
  - 一般我们将低2GB规定为用户态空间，即一般的应用程序可以控制的空间
  - 高2GB的空间属于内核态空间，这部分空间是由操作系统控制的，用户态程序不能控制



32位MIPS的内存分配

- 应用程序中常见的数据

```
int global;

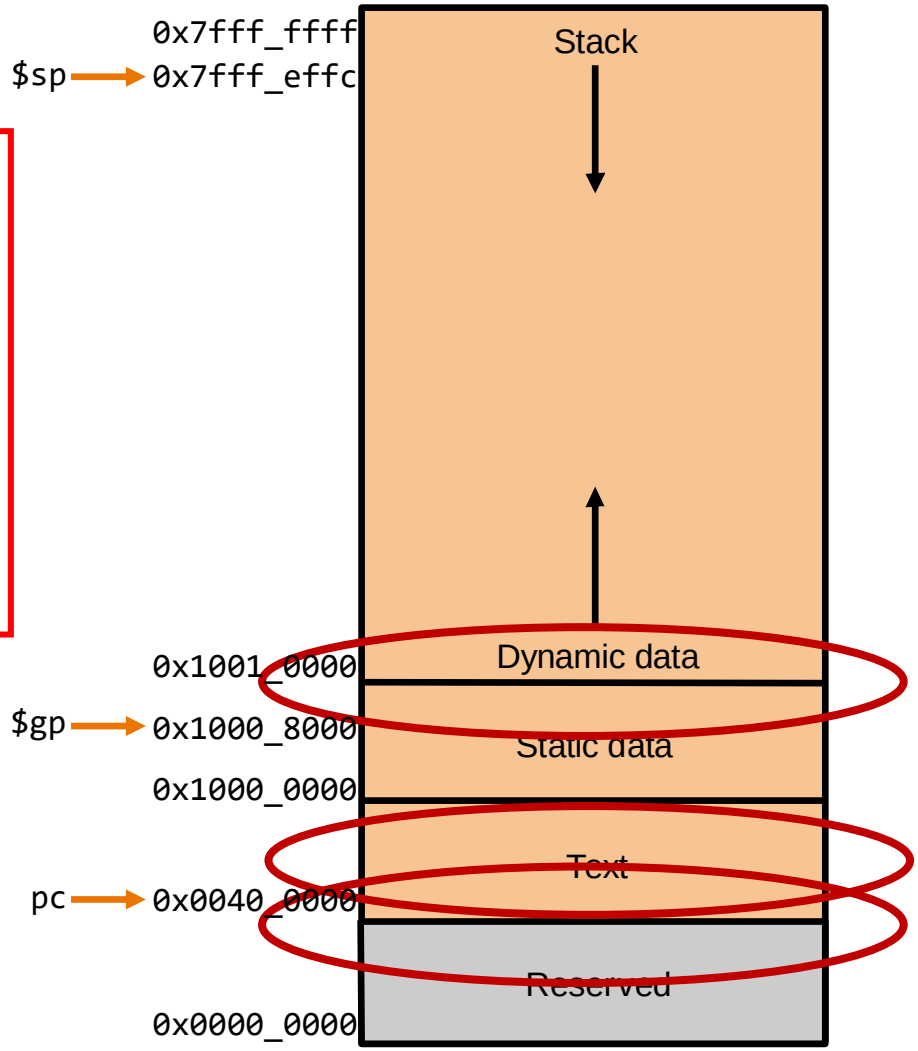
int main()
{
    int local;
    char array0[10];
    int * array1;
    array1=(int *) malloc(10*sizeof(int));
    free(array1);
}
```

程序运行时这些数据都被放在内存空间的什么地方呢？

32位MIPS的内存分配

- 在底端，程序代码（文本）开始于0x00400000；
- 静态数据开始于0x10000000；
- 紧接着是由C中malloc进行存储器分配的动态数据，朝堆栈段向上增长；
- 从顶端开始，对栈指针初始化为0x7fffeffc，并向下向数据段增长

全局指针被设定为易于访问数据的地址，以便使用相对于$gp的±16位偏移量 0x1000_0000 - 0x1000_ffff



MIPS汇编：语法

- 注释以“#” 开始；
- 标签（label）由字母、下划线（_）、点（.）构成，但不能以数字开头；标签大小写敏感；指令操作码是一些保留字，不能用作标签；
- 标签放在行首，后跟冒号（:），例如

```
        .data                   # 将子数据项，存放到数据段中
Item:   .word 1,2               # 将2个32位数值送入地址连续的内存字中
        .text                   # 将子串，即指令或字送入用户文件段
        .global main            # 必须为全局变量
Main:   lw $t0, item
```

MIPS汇编：语法

MIPS汇编语言语句格式

- 指令与伪指令语句

```
[label:] <op> [Arg1] [Arg2] ...
  [#comment]
```

例如 `AddFunc: add $a1, $a2, $a3 # a1=a2+a3`

- 汇编命令(directive)语句

```
[label:] .directive  [Arg1] [Arg2] ...
  [#comment]
```

例如 `.word 0xa3`

MIPS汇编：语法

- 指令与伪指令（Pseudo Instructions）
  - 有一些MIPS指令是和机器码一一对应，可以直接翻译成机器码
    - `add $s0, $s1, $s2`　　`lw $t0, 4($t1)`
  - 还有一些没有对应的机器码，不能直接翻译成机器码，需要先翻译成真的指令
    - `li $s1, 0x7f` `<===>` `addiu $s1, $zero, 0x7f`
  - 编写程序时使用伪指令有利于提高效率并增加可读性
- 伪指令可在MARS仿真器（稍后介绍）的“帮助”下的Extended (pseudo) Instructions页面查询

MIPS汇编：语法

汇编命令

- 汇编器用来定义数据段、代码段以及为数据分配存储空间

```
.data [address]              # 定义数据段，[address]为可选的地址
.text [address]              # 定义正文段(即代码段)， [address]为可选的地址
.align n                     # 以2^n字节边界对齐数据，只能用于数据段
.ascii <string>              # 在内存中存放字符串
.asciiz <string>             # 在内存中存放NULL结束的字符串
.word w1, w2, . . . , wn     # 在内存中存放n个字，值为w1，w2，...，wn
.half h1, h2, . . . , hn     # 在内存中存放n个半字，值为h1，h2，...，hn
.byte b1, b2, . . . , bn     # 在内存中存放n个字节，值为b1，b2，...，bn
.space n                     # 在内存中存放n字节的数组
```

MIPS汇编：语法

```
--                  .data 0x10010000   # 以下部分定义数据段（可省略地址）
0x10010000  n:      .word 5            # 0x10010000处存1个字，值为5。后面程序可用n
来访问此地址
0x10010008
            str:    .asciiz “Hello!“   # 0x10010008处存放以’\0’结尾字符串。str用来
--          访问首地址
0x10010010
                    .align 2           # 对齐到4字节。否则下一个地址为0x1001000f,
--          会出错!
0x00400000  arr:    .space 8           # 0x10010010处存8个字节（2个字）。arr用来访
问首地址
0x00400004
                    .text 0x00400000   # 以下部分定义正文段（可省略地址）
0x00400008
                    la $s0, n          # 伪指令la：用于加载相应label的地址到相应寄
0x0040000c  存器
0x00400010
                    la $s1, arr
0x00400014
                    lw $t0, 0($s0)     # 使用访存指令去读写数据段的数据
                    sw $t0, 0($s1)
                    addi $t0, $t0, 1
                    sw $t0, 4($s1)
```

MIPS汇编：语法

```
    --                  .data 0x10010000  # 以下部分定义数据段（可省略地址）
0x10010000      n:      .word 5           # 0x10010000处存1个字，值为5。后面程序可用n来访问此地址
0x10010008      str:    .asciiz “Hello!“  # 0x10010008处存放以’\0’结尾字符串。str用来访问首地址
    --                  .align 2          # 对齐到4字节。否则下一个地址为0x1001000f，会出错！
0x10010010      arr:    .space 8          # 0x10010010处存8个字节（2个字）。arr用来访问首地址
    --                  .text 0x00400000  # 以下部分定义正文段（可省略地址）
0x00400000              la $s0, n         # 伪指令la：用于加载相应label的地址到相应寄存器
0x00400004              la $s1, arr
0x00400008              lw $t0, 0($s0)    # 使用访存指令去读写数据段的数据
0x0040000c              sw $t0, 0($s1)
0x00400010              addi $t0, $t0, 1
0x00400014              sw $t0, 4($s1)
```

MIPS汇编：变量

- **大部分变量存储在主存储器内**（而不是寄存器内）
  - 因为我们通常有很多的变量要存，不止32个
  - 用.space等汇编命令声明在数据段；为了实现功能，用lw语句将变量加载到寄存器中，对寄存器进行操作，然后再把结果sw回去
- **常访问的变量**
  - 让变量在寄存器中保留时间越长越好（lw和sw比寄存器操作要慢得多得多！）
  - 编译器往往可以自动将常访问的变量作为寄存器变量（如C的register存储类型）
  - 本次实验中可以自行决定是否将变量作为register存储类型，不放入主存
- **临时寄存器**
  - 由于一条指令只能采用两个输入，所以必须采用临时寄存器计算复杂的问题，如z=(x+y)+(x-y)，a[i]=a[n-i-1]

MIPS汇编：数组

- 用.space来给数组开辟空间，或者.word开辟给定初始值的数组
  - 在编译时**静态地**开辟n*4 bytes, (n个32-bit 字)
- 使用lw和sw访问数组

```
lw $temp, 0($A)          # temp =A[0];
sw $temp, 8($B)          # B[2]= temp;
```

注意这里$A, $B存的是地址，$temp存的是具体的值

  - 将常数0，8作为地址偏移量
  - 将寄存器$A和$B作为数组中的开始地址(A[]，B[])

MIPS汇编：数组

```
        .data
arr1:   .word 1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17
arr2:   .space 68
        .text
main:   add $s7, $ra, $zero     # 保存返回地址到$s7
         la $s5, arr1            # $s5=&A
        la $s6, arr2            # $s6=&B
        addi $s0, $zero, 0x11   # Size(A)=Size(B)=0x11
loop:   addi $s0, $s0, -1       # 计数
        sll $s1, $s0, 2         # 换算地址
        add $s2, $s1, $s5       # 计算A[]偏移量,送到$s2
        lw $s3, 0($s2)          # 读出A[]中的值
        add $s2, $s1, $s6       # 计算B[]偏移量,送到$s2
        sw $s3, 0($s2)          # 写到B[]中去
        bnez $s0, loop
        add $ra, $zero, $s7     # 返回主调过程
        jr $ra
```

每个数4Bytes, 1 word

功能：将数组A的值依次拷贝到数组B中

MIPS汇编：数组

使用移位操作代替mul和div：因为mul和div一般都比sll和srl慢

- sll by k 等价于 mul by 2^k
- srl by k 等价于 div by 2^k

} 只对无符号数成立，且没有超出数据表示范围

- 对于有符号数用 sra
  - 高位用符号位填充(在2-补码表示情况下)
  - e.g.,

```
# $s0 = -6 = 0b11…11010
srl $s0, $s0, 1     # 0b01…11101
```

×

```
sra $s0, $s0, 1     # 0b11…11101=-3
```

√

MIPS汇编：系统调用

- **系统调用syscall**：使用syscall可以完成包括文件读写，命令行读写（标准输入输出），申请内存等辅助功能。
- 系统调用基本的使用方法是
  1. 向$a*寄存中写入需要的参数（如果有）
  2. 向$v0寄存器中写入需要调用的syscall的编号
  3. 使用syscall指令进行调用
  4. 从$v0中读取调用的返回值（如果有）
- 系统调用的参数与编号可在MARS仿真器（稍后介绍）的“帮助”下的Syscall页面查询

MIPS汇编：系统调用

```
.text
addi $v0, $zero, 5      # 系统调用编号5（整数输入）放入$v0
syscall                 # 系统调用进行整数输入，结果会进入$v0
add $a0, $zero, $v0     # 将系统调用参数（需要打印的整数）放到$a0
addi $v0, $zero, 1      # 系统调用编号1（整数输出）放入$v0
syscall                 # 系统调用进行整数输出，打印$a0内的数
```

MIPS汇编：过程调用

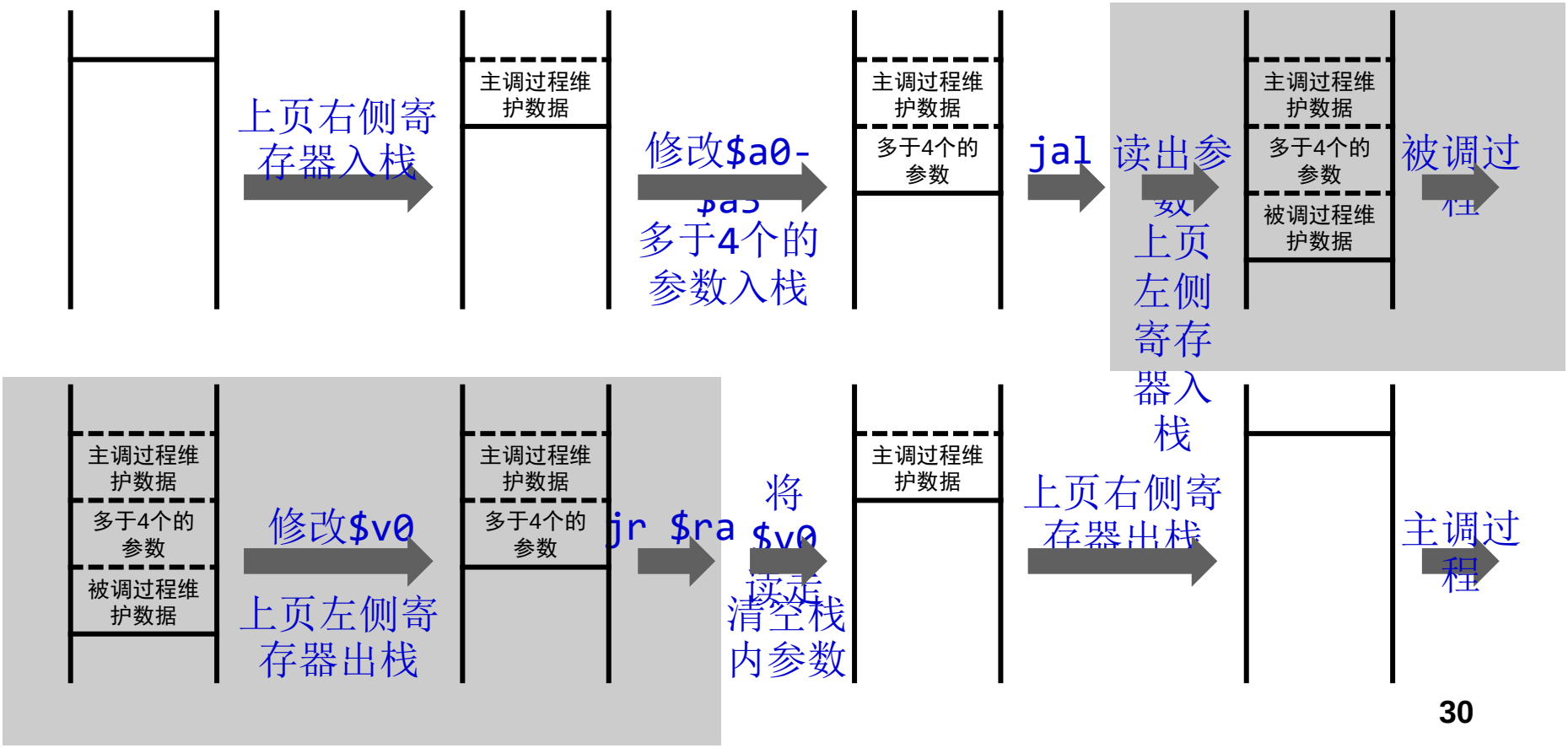
- 过程调用寄存器保持情况（从主调过程的视角）
  - 主调过程和被调过程都需要维护一些数据

被保持（主调过程无需维护）	不被保持
保存寄存器：*$s0-$s7*	临时寄存器：*$t0-$t9*
栈指针：*$sp*	参数寄存器：*$a0-$a3*
返回地址寄存器：*$ra*	返回值寄存器：*$v0-$v1*

- 被调过程不改变这些寄存器数据
- 如果被调过程要用，需要被调过程维护好
- 进入被调过程后存储，离开被调过程前读取

- 被调过程可以改变这些寄存器数据
- 如果主调过程需要调用结束后继续使用，需主调过程提前备份好
- 过程调用前存储，过程调用后读取

MIPS汇编：过程调用


主调过程维护数据
多于4个的参数
被调过程维护数据
上页右侧寄存器入栈
修改$a0-$a3
多于4个的参数入栈
jal
读出参数
上页左侧寄存器入栈
被调过程
修改$v0
上页左侧寄存器出栈
jr $ra
将$v0读走
清空栈内参数
上页右侧寄存器出栈
主调过程

目录

- 汇编程序设计基础
- **MARS环境安装与基础使用方法**
- 实验内容简介
- 参考资料

安装JRE

- 运行JAVA程序包需要运行环境：Java Runtime Environment (JRE)
- 64位Windows系统可运行attachment中jre-8u441-windows-x64.exe进行安装
- 如果安装中又遇到问题，或者其
  访问JAVA官网：https://www.j
  完成后按提示进行安装

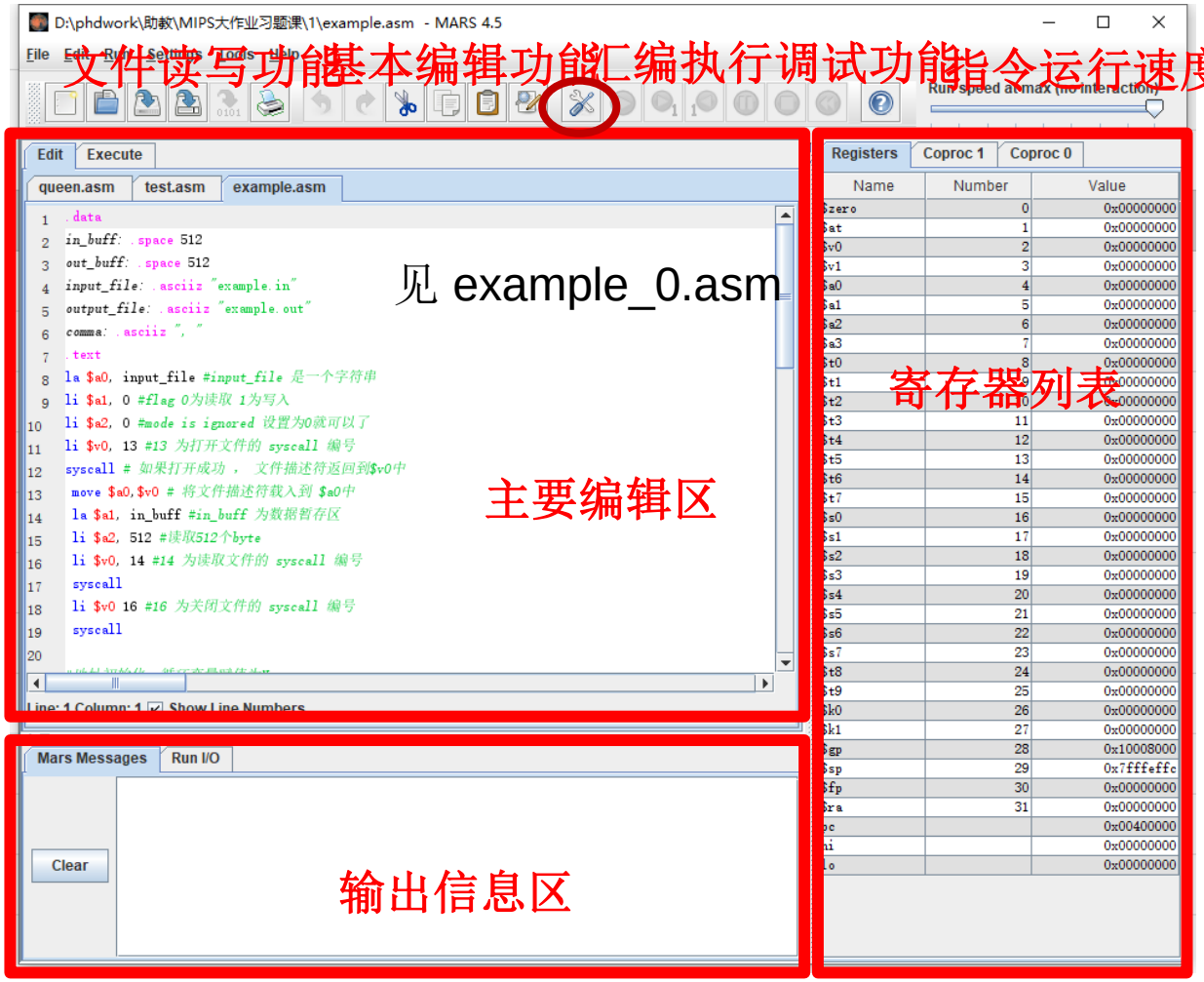


运行MARS

- 如果JRE正确安装，双击Mars4_5.jar即可打开MARS仿真器。此时的工作路径为Mars4_5.jar文件所处路径
- 如果打不开，先检查JRE是否安装正确，可以考虑重新安装
  - 方法：打开命令行，输入java -version并回车，能正确输出版本号即安装正确
  - Java安装正确，可以尝试用java -jar [相应路径/]Mars4_5.jar打开MARS。若成功打开，此时工作路径为命令行当前所在路径
- 如果是MARS软件包的问题可以进入MARS官网下载 https://courses.missouristate.edu/KenVollmar/mars/download.htm
- 如果仍无法解决，请尽早联系助教

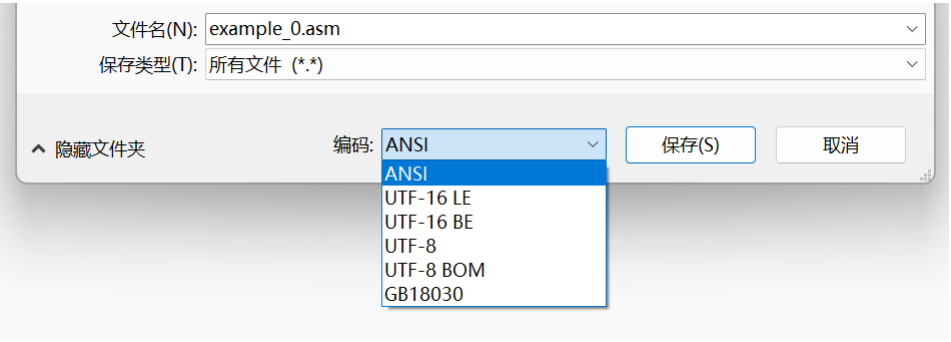
运行MARS

- **主要编辑区**用于编写汇编指令
- **输出信息区**可以查看程序运行过程中的输入输出（Run I/O选项卡）和系统报错（Mars Messages选项卡）等
- **寄存器列表**实时显示当前运行状态下各个寄存器存储的值



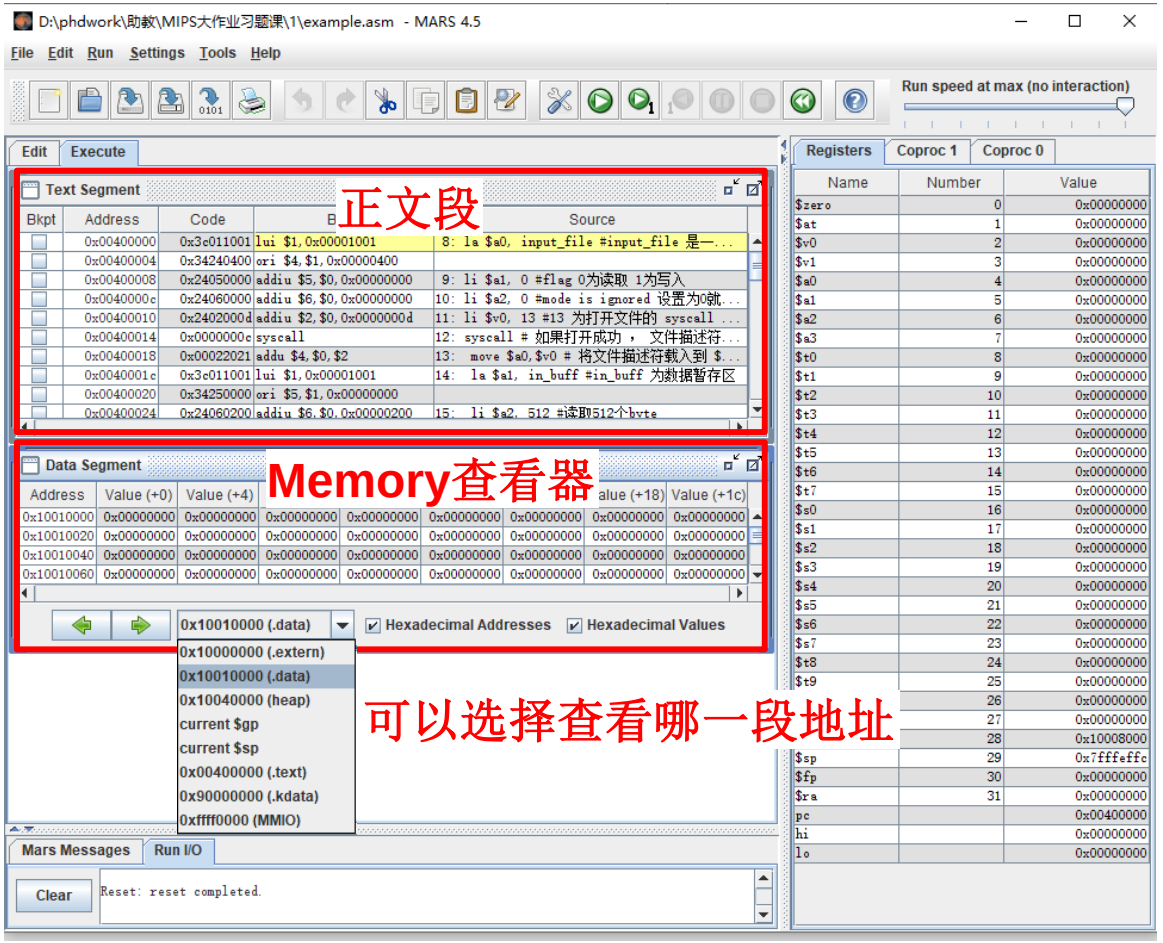
汇编运行

- 首先打开汇编文件example_0.asm，并将输入文件example.in放在工作路径下
- Windows下中文乱码解决方法：用记事本打开后“另存为”，下面编码改为“ANSI”（如右图）



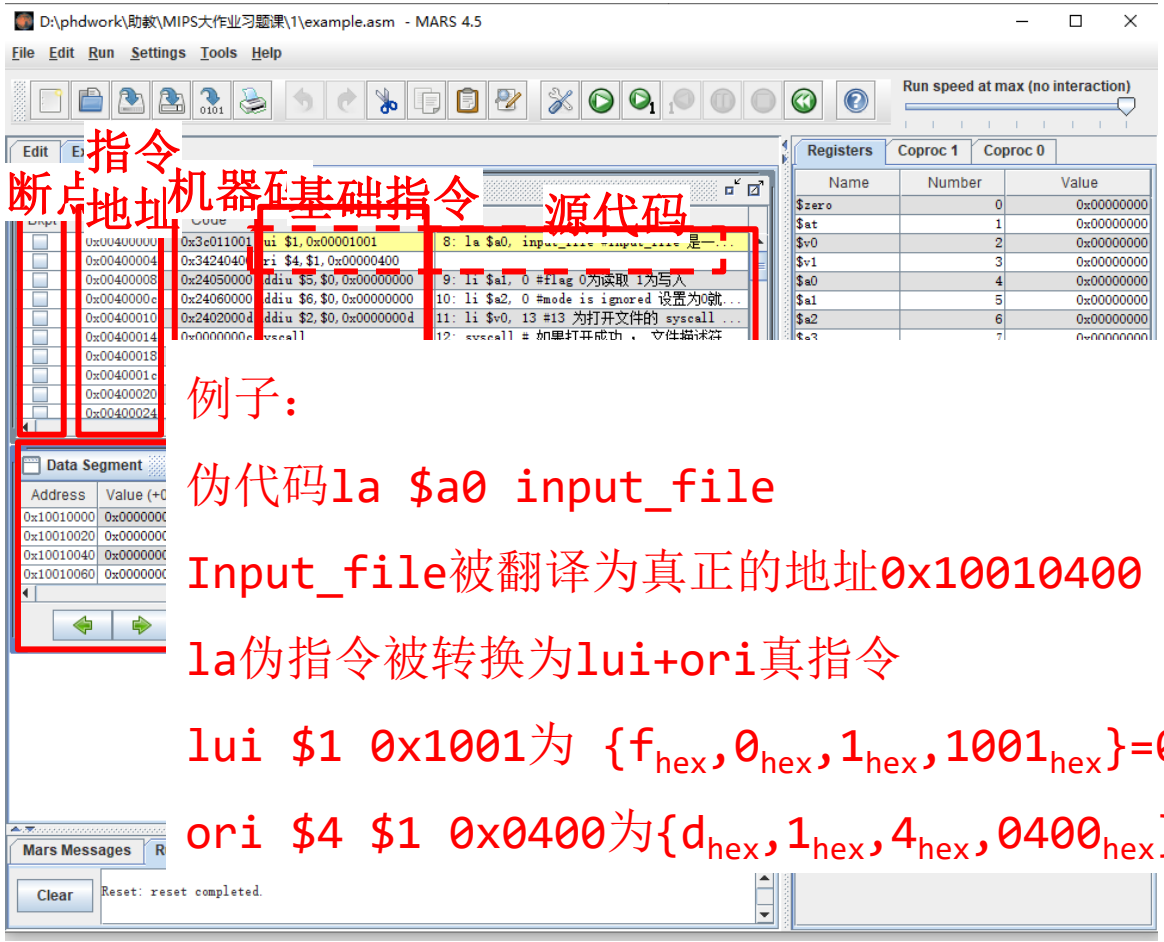
汇编运行

- 点击汇编按钮即可切换到执行页面，源代码汇编成基础指令和机器码，PC置为0x00400000，并等待执行
- 执行页面内可以看到汇编后的基础指令和对应的机器码，每条指令的指令地址



汇编运行

- 源代码：用户编写的汇编代码，包括标记，伪代码等
- 基础指令：汇编后的指令，伪代码被转换，标记被翻译
- 地址&机器码：与基础指令一一对应，32bit一条指令，地址依次加四
- 断点：调试用，当执行到这一句时暂停



例子：

伪代码la $a0 input_file

Input_file被翻译为真正的地址0x10010400

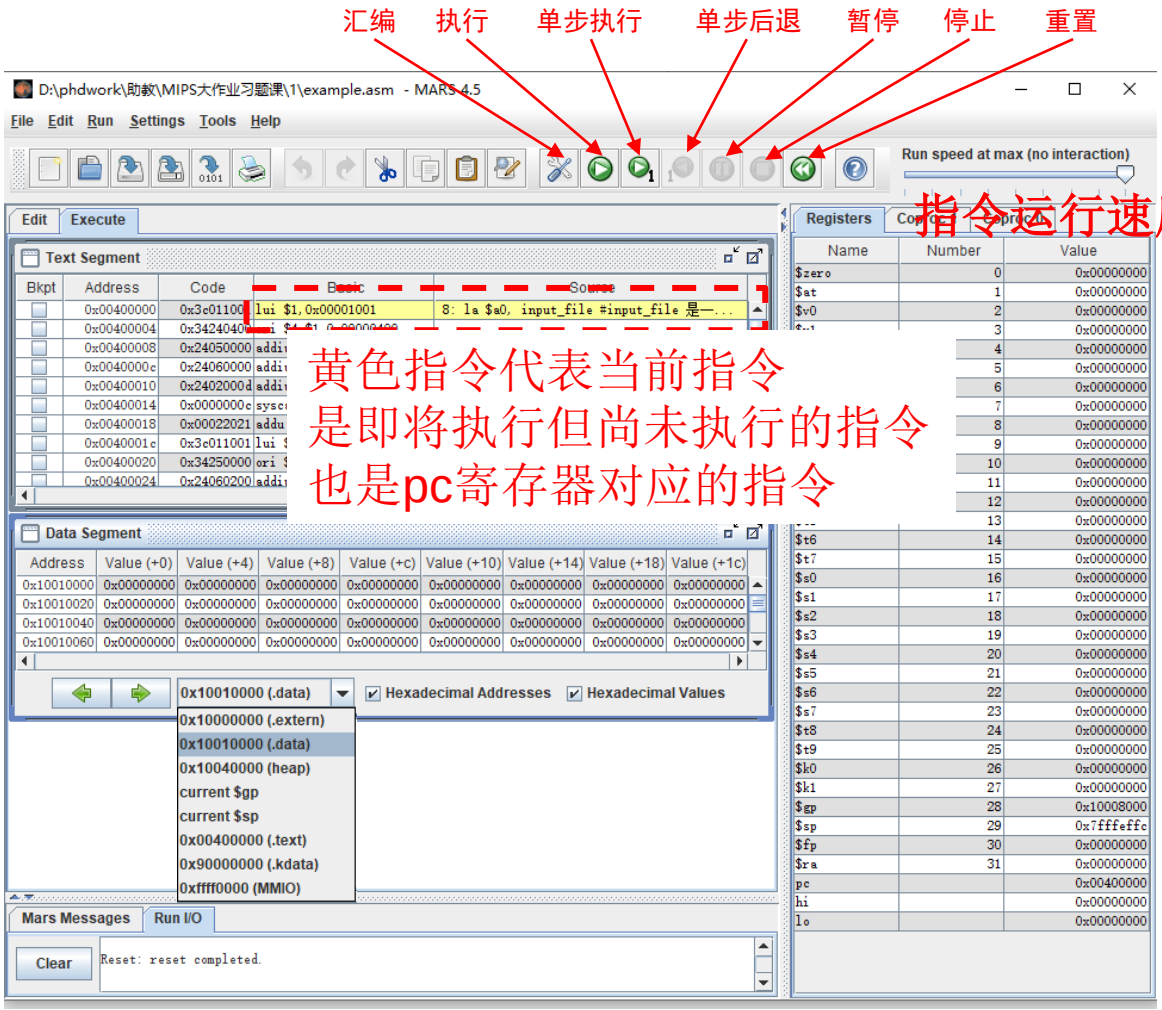
la伪指令被转换为lui+ori真指令

lui $1 0x1001为 $\{f_{hex},0_{hex},1_{hex},1001_{hex}\}=0x$

ori $4 $1 0x0400为$\{d_{hex},1_{hex},4_{hex},0400_{hex}\}=$

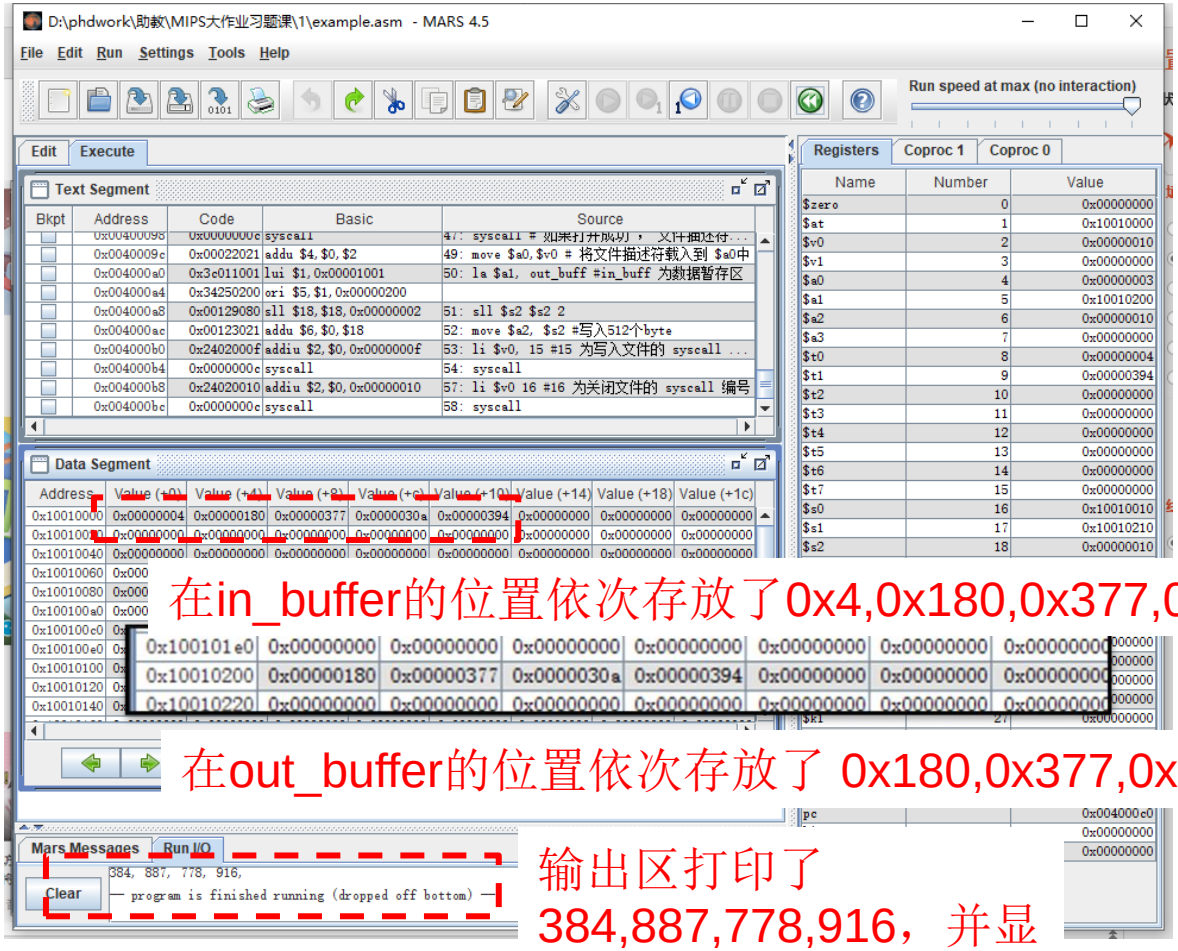
汇编运行

- 执行：从第一条指令开始连续执行直到结束
- 单步执行：执行当前指令并跳转到下一条
- 单步后退：后退到最后一条指令执行前的状态（包括寄存器和memory）
- 暂停&停止：在连续执行的时候可以停下来，一般配合较慢的指令运行速度，不用于调试。**调试最好使用断点功能**
- 重置：重置所有寄存器和memory



汇编运行

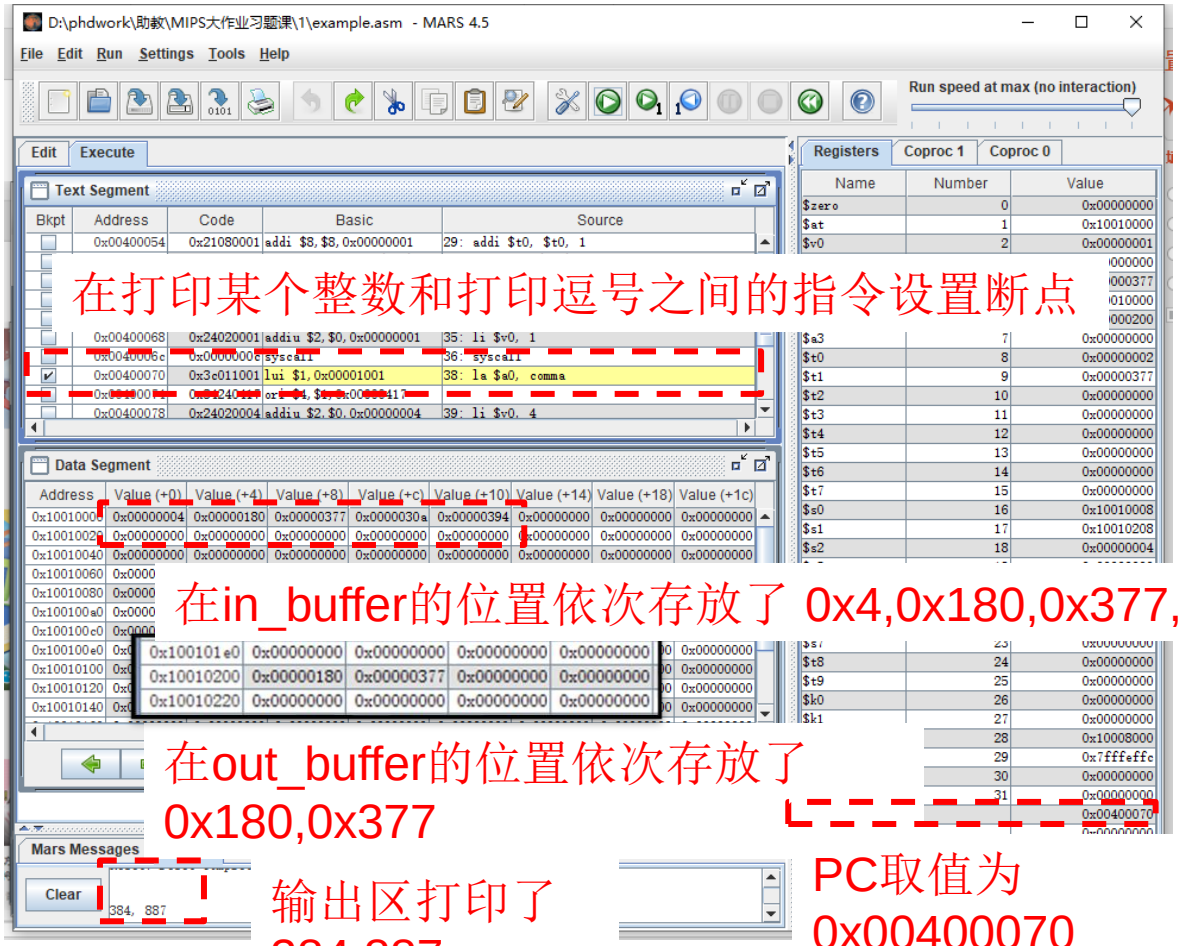
- 点击执行按钮后，所有指令执行完毕
- 各个寄存器内的值发生了变化，内存中in_buffer和out_buffer地址对应的数据发生变化。
- 输出区正确打印了对应的数据并提示，程序执行完地址最大的指令并且没有后续指令了（dropped off bottom）。



汇编运行

```
38 la $a0, comma
```

- 在38行的指令处设置断点。并点击运行按钮，程序停在该位置
- 运行两次后，可以看到程序向out_buffer中写入两个数，也向输出区打了两个数，各个寄存器也停留在对应状态



汇编运行

example_0.asm 内包含一个从文件读取数据并写入另一个文件的例子

数据声明，此部分数据存在0x10

```
.data
in_buff: .space 512
out_buff: .space 512
input_file: .asciiz "example.in"
output_file: .asciiz "example.out"
comma: .asciiz ", "
```

```
.text
la $a0, input_file #input_file 是一个字符串
li $a1, 0 #flag 0为读取 1为写入
li $a2, 0 #mode is ignored 设置为0就可以了
li $v0, 13 #13 为打开文件的 syscall 编号
syscall # 如果打开成功 ， 文件描述符返回到
move $a0,$v0 # 将文件描述符载入到 $a0中
la $a1, in_buff #in_buff 为数据暂存区
li $a2, 512 #读取512个byte
li $v0, 14 #14 为读取文件的 syscall 编号
syscall
li $v0 16 #16 为关闭文件的 syscall 编号
syscall
```

打开读取文件，并将数据写入in_buff

初始化变量

```
la $s1, out_buff
lw $s2, 0($s0)
li $t0, 0
```

循环体

```
for: addi $s0, $s0, 4
addi $t0, $t0, 1
lw $t1, 0($s0)
sw $t1, 0($s1)
addi $s1, $s1, 4
#打印整数
move $a0, $t1
li $v0, 1
syscall
#打印逗号
la $a0, comma
li $v0, 4
syscall
bne $t0, $s2
```

跳转条件

注意被读取文件需要放在工作路径下这个例程才能正常读取

打开文件并将out_buff的数据写

```
la $a0, output_file #output_file 是一个字符串
li $a1, 1 #flag 0为读取 1为写入
li $a2, 0 #mode is ignored 设置为0就可以了
li $v0, 13 #13 为打开文件的 syscall 编号
syscall # 如果打开成功 ， 文件描述符返回到$v0中

move $a0,$v0 # 将文件描述符载入到 $a0中
la $a1, out_buff #out_buff 为数据暂存区
sll $s2 $s2 2
move $a2, $s2 #写入N个byte
li $v0, 15 #15 为写入文件的 syscall 编号
syscall
#此时$a0 中的文件描述符没变
#直接调用 syscall 16 关闭它
li $v0 16 #16 为关闭文件的 syscall 编号
syscall
```

其他样例代码简介

- **example_1.asm**
  - **基本代码格式**
- **example_2.asm**
  - **汇编命令使用**
- **example_3.asm**
  - **函数使用（无出入栈）**

见 example_3.asm

```
product:
    move $t0 $a0  #将第一个参数赋给t0作为累加值
    move $t1 $a1  #将第二个参数赋给t1作为计数器
    move $t2 $zero#结果清零
    loop: add $t2 $t2 $t0  #结果累加t0
          addi $t1 $t1 -1  #计数器减一
          bnez $t1 loop    #如果计数器不为零循环继续
    move $v0 $t2  #将结果赋给返回值
    jr $ra        #跳转回上一级程序
```

目录

- 汇编程序设计基础
- MARS环境安装与基础使用方法
- **实验内容简介**
- 参考资料

作业部分1

- 用MIPS32汇编指令完成下列任务，调试代码并获得正确的结果
  - 练习1-1：系统调用
  - 练习1-2：循环，分支
  - 练习1-3：数组，指针
  - 练习1-4：函数调用

作业部分2

- 用MIPS32汇编指令翻译给定的代码解决对应题目
- 翻译要求：不要求逐句对应，但程序执行整体流程应当和所给出的代码**完全一致**（如C程序中的分支、循环、过程调用，**必须**在汇编代码中做出相应动作）

作业提交要求

- DDL：2026年6月14日（16周周日）晚23:59
- 详细提交要求见说明文档
  - 需要使用MIPS32汇编对给定C或C++程序进行翻译，不可使用其他汇编语言，不可自行另写程序
  - 未按要求进行输入输出/提交可能会导致脚本评测失分！
  - 实验报告：只需要给出运行结果&让助教能结合注释看懂代码
- 不要造假，不要抄袭！！！
  - 会查重（同届和往届代码，网络代码，AI助手生成代码等）
  - 若使用AI辅助完成题目，必须明确标注本题由 AI 辅助完成，并说明AI提供商及模型名称（例如：OpenAI-GPT-5 Mini）。同时，需在附录中以截图形式提供与AI的完整对话记录。

目录

- 汇编程序设计基础
- MARS环境安装与基础使用方法
- 实验内容简介
- **参考资料**

参考资料

- 指令集架构简介 *Introduction to the MIPS32 Architecture.pdf*
- 指令集手册 *MIPS32 Instruction Set Manual.pdf*
- 课程教材第V章
- 指令集课件附录

附件

- JAVA运行环境安装包（64位Windows系统）jre-8u441-windows-x64.exe
  - 其他系统JAVA运行环境参考官网https://www.java.com/
- MARS模拟器 Mars4_5.jar
- 二进制文件的查看（binaryview.c）和比较（compare.c）
  - 自行编译即可使用
  - binaryview查看32位整数：[编译后可执行文件] [待查看文件] [输出文件] -i
  - compare：[编译后可执行文件] [文件1] [文件2]